

Annexure A — Design Task Solutions

Hybrid AI Systems on PySpark: Distributed Processing & Intelligent Agents

CO5-AT1\CSA1711-ARTIFICIAL INTELLIGENCE

1. Hybrid AI Recommendation Engine Design

1.1 Problem Overview & Objective

The goal is to design a hybrid recommendation engine that combines content-based filtering (matching item attributes to a user's known preferences) with collaborative filtering (matching a user to similar users/items based on interaction patterns). The hybrid approach overcomes the individual weaknesses of each method — content-based filtering struggles with novelty and cross-category discovery, while collaborative filtering suffers from the cold-start problem for new users or items. PySpark RDDs provide the distributed backbone needed to process millions of user-item interactions at scale.

1.2 System Architecture

Layer	Component	Technology / Technique
Ingestion	Clickstream, ratings, purchase logs, item metadata	Kafka / Spark Structured Streaming
Storage	Raw interaction logs, item catalog	HDFS / S3 / Delta Lake
Processing	RDD transformations for cleaning, joining, feature extraction	PySpark Core (RDD API)
Content-Based Module	TF-IDF / embedding similarity on item attributes	Spark MLlib, cosine similarity RDD maps
Collaborative Module	User-item matrix factorization	Spark MLlib ALS (Alternating Least Squares)
Hybridization Layer	Weighted / switching / stacking ensemble of both scores	Custom PySpark scoring pipeline
Agent Layer	Adaptive re-ranking, feedback capture, exploration	Multi-agent reinforcement learning
Serving	Low-latency recommendation API	Spark Streaming + REST microservice / cache

1.3 Distributed Data Processing with PySpark RDDs

Large-scale user-item interaction data is naturally suited to RDDs because it is record-oriented, partitionable, and requires repeated map/reduce style aggregation:

- Partitioning: interaction logs are partitioned by user ID (hash partitioning) so that all of a user's history sits on the same executor, minimizing shuffle during profile construction.
- Map transformations: raw logs are mapped into (user_id, item_id, rating/implicit_score) tuples; item metadata is mapped into TF-IDF feature vectors.
- ReduceByKey / GroupByKey: user-item ratings are aggregated to build sparse user vectors and item co-occurrence counts needed for collaborative filtering.
- Join operations: RDD joins combine the user-item matrix with item content vectors so the hybrid scorer can access both signals per record without re-reading source data.

- Caching/persistence: intermediate RDDs (e.g., the trained ALS factor matrices, TF-IDF item vectors) are cached in memory since they are reused across many scoring requests.

```
interactions = sc.textFile("hdfs://logs/interactions/*")
pairs = interactions.map(lambda l: l.split(",")\
    .map(lambda r: (r[0], (r[1], float(r[2])))))
user_profiles = pairs.groupByKey().mapValues(build_profile)
item_vectors = sc.textFile("hdfs://catalog/items/*")\
    .map(parse_item).map(lambda i: (i.id, tfidf_vector(i)))
hybrid_scores = user_profiles.join(item_vectors)\
    .flatMap(lambda kv: score_hybrid(kv))
hybrid_scores.persist()
```

1.4 Role of Intelligent Agents in Real-Time Adaptation

Intelligent agents sit above the batch-trained models and handle the online, behavioral side of personalization:

- Monitoring Agent: continuously observes click-through, dwell time, and skip signals as a real-time stream and updates a short-term user context vector.
- Re-ranking Agent: applies a bandit algorithm (e.g., contextual multi-armed bandit or Thompson Sampling) to blend the batch hybrid score with the live context, balancing exploitation of known preferences against exploration of new items.
- Feedback Agent: closes the loop by writing observed outcomes (accept/reject) back into the training stream, which is periodically reprocessed by the PySpark batch job to refresh ALS factors and content vectors.
- Coordinator Agent: negotiates between the content-based and collaborative agents when their recommendations diverge, using a confidence-weighted voting scheme.

This agent layer converts a static offline model into a system that adapts within a session, not just between retraining cycles.

2. Smart Disaster Detection System

2.1 Problem Overview & Objective

The system must fuse heterogeneous, high-volume data streams — satellite imagery, IoT/environmental sensors, and social media text — to detect early signs of floods, earthquakes, or other natural disasters, and generate timely, geographically-targeted warnings.

2.2 System Architecture

Layer	Component	Technology / Technique
Data Sources	Satellite feeds (optical/SAR), seismic & water-level sensors, Twitter/social streams	Kafka Connect, sensor gateways, social APIs
Ingestion	Multi-source stream ingestion with source tagging	Spark Structured Streaming
Distributed Processing	Parallel cleaning, transformation, feature extraction per source	PySpark RDD / DataFrame transformations
Data Fusion Layer	Spatio-temporal alignment and weighted fusion of signals	Custom fusion RDD pipeline, Kalman/Bayesian fusion
ML/Anomaly Layer	Image change-detection, sensor threshold + anomaly models, NLP on social text	Spark MLlib, CNN (offline-trained), sentiment/NER
Agent Layer	Detection, verification, alerting and response-coordination agents	Multi-agent system (MAS)
Warning & Response	Geo-targeted alert dispatch, dashboards for responders	REST/SMS/cell-broadcast gateway

2.3 Distributed Processing & Data Fusion with PySpark RDDs

- Source-specific RDDs: each stream (imagery tiles, sensor readings, social posts) is loaded into its own RDD and transformed independently — image tiles into change-intensity scores, sensor readings into normalized time-series, tweets into geo-tagged sentiment/keyword features.
- Spatio-temporal keying: every record is re-keyed to a common (geohash, time_bucket) key using map transformations, which is what makes fusion possible across otherwise incompatible data types.
- CoGroup / Join fusion: RDD cogroup() brings together all signals sharing a (geohash, time_bucket) key, producing a combined feature vector per grid cell per time window.
- Weighted aggregation: reduceByKey combines the multi-source features into a single risk score per cell, with fusion weights that reflect each source's historical reliability for the disaster type.

```
sat = sc.textFile("satellite/*").map(parse_tile).map(lambda t: ((t.geohash, t.tbucket), ("sat", t.change_score)))
sens = sc.textFile("sensors/*").map(parse_sensor).map(lambda s: ((s.geohash, s.tbucket), ("sensor", s.zscore)))
soc = sc.textFile("social/*").map(parse_post).map(lambda p: ((p.geohash, p.tbucket), ("social", p.risk_kw_score)))

fused = sat.union(sens).union(soc)\
    .groupByKey()\
    .mapValues(fuse_weighted) # -> single risk score per cell/time
alerts = fused.filter(lambda kv: kv[1] > THRESHOLD)
```

2.4 Role of Intelligent Agents in Coordination & Early Warning

- Detection Agents (per data source): each runs local anomaly detection (e.g., a sensor agent flags rapid water-level rise; an imagery agent flags flooded-area growth) and publishes a confidence-scored event.
- Fusion/Verification Agent: cross-checks whether independent agents corroborate the same geolocation and time window before escalating, reducing false positives from any single noisy source.
- Alerting Agent: once verified, determines severity tier and target region, then triggers geo-targeted notifications (SMS, cell broadcast, dashboards for emergency responders).
- Response-Coordination Agent: interfaces with logistics/evacuation planning, suggesting resource allocation based on population density and predicted disaster spread.
- Learning loop: outcomes (confirmed disaster vs. false alarm) are fed back so agents' confidence-weighting and fusion thresholds are periodically retrained.

3. AI-Based Fraud Detection Framework

3.1 Problem Overview & Objective

Financial fraud detection requires scanning very large, high-velocity transaction volumes for rare, evolving anomalous patterns, while keeping false-positive rates low enough not to block legitimate customers. The system must detect in near real time and continuously adapt as fraud tactics change.

3.2 System Architecture

Layer	Component	Technology / Technique
Ingestion	Real-time transaction stream (card swipes, transfers, logins)	Kafka / Spark Structured Streaming
Distributed Processing	Feature engineering across billions of transactions	PySpark RDD transformations
Anomaly Detection Layer	Unsupervised + supervised scoring per transaction	Isolation Forest, clustering, gradient-boosted classifier (Spark MLlib)

Graph Layer	Entity-relationship analysis (accounts, devices, merchants)	GraphFrames / GraphX on Spark
Agent Layer	Monitoring, investigation, and adaptive-learning agents	Multi-agent system
Alerting & Case Mgmt	Analyst dashboards, automatic holds	REST APIs, case-management system

3.3 Distributed Anomaly Detection with PySpark RDDs

- Feature extraction RDDs: each transaction record is mapped into a feature vector (amount deviation from account baseline, transaction velocity, merchant-category risk, geolocation jump distance) using per-account historical aggregates computed via `reduceByKey`.
- Windowed aggregation: transactions are grouped into sliding time windows per account using key-value RDD operations, enabling detection of bursts (e.g., many small transactions in a short period — a classic card-testing pattern).
- Distributed anomaly scoring: an Isolation Forest / statistical z-score model is applied via `mapPartitions` so each partition scores its transactions in parallel without shuffling the whole dataset.
- Graph-based collusion detection: RDDs of (account, counterparty) edges are used to build a transaction graph; connected-component and PageRank-style analysis surfaces rings of colluding accounts.

```
tx = sc.textFile("transactions/*").map(parse_tx)
keyed = tx.map(lambda t: (t.account_id, t))
baselines = keyed.groupByKey().mapValues(compute_baseline) # avg, std, velocity
scored = keyed.join(baselines).mapPartitions(score_partition) # anomaly score per tx
flagged = scored.filter(lambda kv: kv[1].score > RISK_THRESHOLD)
```

3.4 Role of Intelligent Agents

- Monitoring Agents: attached to account/merchant segments, continuously watch the streaming score feed for their segment and raise a local alert when a threshold is crossed.
- Correlation Agent: aggregates alerts across accounts/devices to distinguish an isolated anomaly from a coordinated fraud ring, using the graph layer output.
- Decision Agent: applies policy rules (e.g., auto-hold vs. step-up authentication vs. analyst review) balancing fraud risk against customer friction.
- Learning Agent: feeds analyst-confirmed outcomes (true fraud / false positive) back into the training pipeline, triggering periodic PySpark MLlib model retraining so detection accuracy improves as fraud patterns evolve.

4. Autonomous Supply Chain Optimization System

4.1 Problem Overview & Objective

The objective is to optimize inventory levels, logistics routing, and demand forecasting across a distributed supply network, using decentralized intelligent agents that each make local decisions while a PySpark-based analytics layer processes the large-scale historical and real-time data needed to inform those decisions.

4.2 System Architecture

Layer	Component	Technology / Technique
Data Sources	POS sales, warehouse inventory, GPS/logistics telemetry, supplier feeds	IoT gateways, ERP exports
Distributed Processing	Cleaning, aggregation, feature engineering at scale	PySpark RDD/DataFrame transformations

Forecasting Layer	Demand forecasting per SKU/region	Spark MLlib time-series / gradient boosting
Optimization Layer	Inventory reorder points, route optimization	Linear/integer programming, heuristic solvers
Agent Layer	Warehouse, transport, and procurement agents negotiating decentralized decisions	Multi-agent system (contract-net protocol)
Execution	Automated purchase orders, route dispatch	ERP/TMS integration APIs

4.3 RDD Transformations for Large-Scale Supply Chain Data

- Map/filter for cleaning: raw ERP and IoT records are mapped into a canonical schema and filtered for validity (e.g., discarding duplicate GPS pings).
- ReduceByKey for demand aggregation: per-SKU, per-store sales are aggregated into daily/weekly demand series, which feed the forecasting models.
- Join for landed-cost calculation: inventory RDDs are joined with supplier-pricing and logistics-cost RDDs to compute true landed cost per unit per route, which the optimization layer uses as its objective function input.
- FlatMap for route candidate generation: warehouse-to-store distance/time RDDs are expanded into candidate route options that a downstream optimizer (e.g., linear programming solver) selects from.

```
sales = sc.textFile("pos/*").map(parse_sale)
demand = sales.map(lambda s: ((s.sku, s.store, s.week), s.qty))\
               .reduceByKey(lambda a, b: a + b)
inv     = sc.textFile("inventory/*").map(parse_inv).map(lambda i: (i.sku, i))
cost    = sc.textFile("logistics/*").map(parse_cost).map(lambda c: (c.sku, c))
landed  = inv.join(cost).mapValues(compute_landed_cost)
```

4.4 Decentralized Intelligent Agents

- Inventory Agents (per warehouse): monitor local stock against the PySpark-generated forecast and reorder points, and autonomously trigger replenishment orders.
- Transport Agents (per vehicle/route): negotiate with warehouse agents via a contract-net protocol — warehouses broadcast shipment needs, transport agents bid based on capacity and cost, and the best bid wins the job without central dispatch.
- Procurement Agents: interact with supplier systems, balancing price, lead time, and reliability, escalating to a human planner only when constraints conflict.
- Coordinator/Arbitration Agent: resolves conflicts (e.g., two warehouses competing for the same limited stock) using priority rules derived from service-level commitments.

This decentralization means no single point of failure controls the whole network — agents make local, fast decisions, while the PySpark analytics layer supplies them with globally-consistent forecasts and cost data on a scheduled or streaming basis.

5. Personalized Learning Analytics Platform

5.1 Problem Overview & Objective

The platform must analyze behavioral and performance data (video-watch patterns, quiz attempts, time-on-task, forum activity) from thousands of concurrent learners, and use that analysis to adapt content delivery in real time — e.g., adjusting difficulty, recommending remedial material, or pacing lessons per learner.

5.2 System Architecture

Layer	Component	Technology / Technique
-------	-----------	------------------------

Data Sources	LMS clickstream, quiz/assessment results, video engagement, forum posts	LMS event logs, xAPI/SCORM
Ingestion	Real-time learner-event stream	Kafka / Spark Structured Streaming
Distributed Processing	Behavior aggregation, mastery estimation at scale	PySpark RDD transformations
Modeling Layer	Knowledge-tracing / mastery models, at-risk prediction	Spark MLlib (classification/regression), Bayesian Knowledge Tracing
Agent Layer	Tutoring, pacing, and intervention agents	Multi-agent adaptive-learning system
Delivery	Personalized content sequencing, dashboards for instructors	Adaptive content API, instructor dashboard

5.3 Scalable Analytics with PySpark RDDs

- Per-learner event RDDs: raw interaction events are mapped into (student_id, activity_id, outcome, timestamp) tuples and partitioned by student_id so each learner's full history processes together.
- Mastery aggregation: reduceByKey/groupByKey aggregates quiz results per topic per learner into a mastery score, which is the core signal for adaptive sequencing.
- Cohort-level analytics: the same event RDDs are re-keyed by (course_id, topic) and aggregated to compute cohort-wide difficulty indicators — topics where many learners struggle, useful for instructors.
- Join with content metadata: mastery RDDs are joined with a content-difficulty-tagged catalog RDD so the recommendation step can match a learner's current level to appropriately challenging material.

```
events = sc.textFile("lms_events/*").map(parse_event)
by_student = events.map(lambda e: (e.student_id, e))
mastery = by_student.groupByKey().mapValues(estimate_mastery) # per-topic mastery vector
content = sc.textFile("catalog/*").map(parse_content).map(lambda c: (c.topic, c))
plan = mastery.flatMap(lambda kv: [(kv[0], topic, level) for topic, level in kv[1].items()])\
    .map(lambda t: (t[1], (t[0], t[2])))\
    .join(content)
```

5.4 Adaptive Learning via Intelligent Agents

- Diagnostic Agent: continuously updates each learner's mastery/knowledge state as new interaction events arrive, using the PySpark-computed baseline plus incremental streaming updates.
- Tutoring/Pacing Agent: selects the next piece of content or difficulty level for each learner in real time, balancing challenge against frustration (a reinforcement-learning policy trained on historical engagement outcomes).
- Intervention Agent: flags at-risk learners (e.g., declining engagement, repeated failures on a topic) and triggers remedial resources or an instructor notification.
- Feedback Agent: captures whether adaptive recommendations improved outcomes (e.g., a learner's mastery increased after a suggested resource) and writes that signal back into the training data for periodic model refresh.

Together, the PySpark batch/streaming layer supplies scalable, statistically robust learner models, while the agent layer delivers the moment-to-moment personalization that makes the platform feel adaptive to each of thousands of concurrent learners.

Common Design Pattern Across All Five Systems

Each solution follows the same underlying pattern: (1) PySpark RDDs provide fault-tolerant, partitioned, in-memory distributed processing for feature engineering and model training at scale; (2) a fusion/aggregation layer combines heterogeneous or multi-source signals into a unified representation; and (3) a layer of intelligent agents operates on top of the batch-trained models to deliver real-time, decentralized, adaptive decision-making, closing the loop by feeding observed outcomes back into the next PySpark retraining cycle.